

Position: Multi-Agent Systems Should Prioritize Concurrency Control

Xin Yang^{*1} Letian Li^{*2} Zimo Ji³ Terry Jingchen Zhang^{†4} Wenyuan Jiang^{†4}

Abstract

LLM-based multi-agent systems (MAS) promise scalable collaboration, yet adding agents often *reduces* reliability. This position paper argues that many MAS failures are fundamentally **concurrency control problems**: agents concurrently read and write shared state, and long LLM inference windows amplify the risk of stale reads, lost updates, and inconsistent outcomes. Failure modes commonly attributed to “coordination” or “communication” breakdowns can be mapped directly onto classical concurrency anomalies. We contend that MAS frameworks should address these failures through explicit concurrency control mechanisms: conflict detection, isolation guarantees, and structured access to shared resources. Concurrency control should be a first-class design concern, not an afterthought.

1. Introduction

Large language models (LLMs) have demonstrated increasingly powerful capabilities in language understanding, reasoning, and generation (Brown et al., 2020; OpenAI, 2023; Touvron et al., 2023; Yang et al., 2026), expanding their application domains from text generation to complex decision-making. The integration of tool calling and environmental interaction has further transformed LLMs into autonomous agents capable of executing real-world tasks through iterative reasoning and action (Yao et al., 2023; Schick et al., 2023; Mialon et al., 2023; Cheng et al., 2025). This paradigm shift, exemplified by the ReAct framework (Yao et al., 2023), enables models to move beyond pure dialogue toward practical task completion through observation, reasoning, and action loops. More recently, the pursuit of

^{*}Equal contribution ¹School of Mathematical Sciences, Zhejiang University, Hangzhou, China ²Shenzhen International Graduate School, Tsinghua University, Shenzhen, China ³Hong Kong University of Science and Technology, Clear Water Bay, Hong Kong ⁴ETH Zurich, Zurich, Switzerland. Correspondence to: Wenyuan Jiang <wenyuanjiang@ethz.ch>.

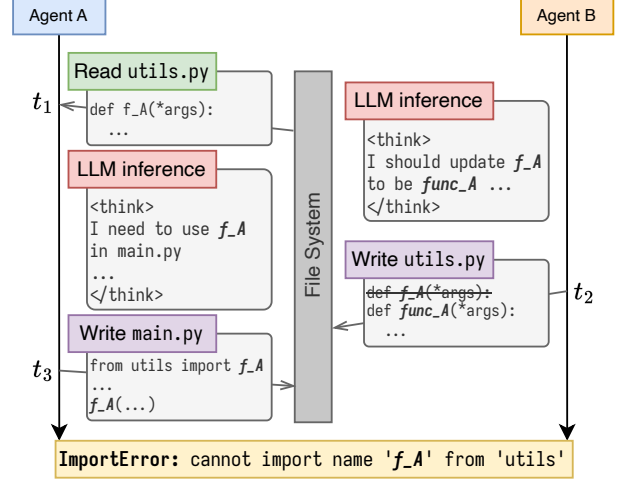


Figure 1. Stale read hazard in multi-agent coding. Agent A reads `utils.py` and enters a long inference phase while implementing `main.py`. Concurrently, Agent B refactors `utils.py`, renaming `f_A` into `func_A`. Both agents act correctly in isolation, yet the interleaving yields a broken import, a classic concurrency anomaly amplified by long LLM inference windows.

solving increasingly complex tasks has driven the development of multi-agent systems (MAS), where multiple agents operate *concurrently*, communicating and collaborating to achieve goals at greater scale and with higher success rates than single-agent approaches (Park et al., 2023; Li et al., 2023; Hong et al., 2024; Qian et al., 2024; Wu et al., 2023; Guo et al., 2024; Anonymous, 2026). However, empirical evidence reveals a sobering reality: scaling the number of agents does not reliably improve performance (Cemri et al., 2025; Kim et al., 2025). Studies show that multi-agent systems exhibit failure rates between 41% and 86.7% across popular benchmarks, with coordination failures and inter-agent misalignment accounting for a substantial fraction of these breakdowns (Cemri et al., 2025; Li et al., 2024a). Recent systems and benchmarks provide concrete evidence that concurrency mechanisms affect MAS outcomes (Geng & Neubig, 2026; Chen et al., 2024a; Zhang et al., 2026; Wang et al., 2024b; Chang & Geng, 2025).

In this position paper, we argue that many communication and coordination failures in MAS are fundamentally

concurrency control problems, which should be considered as a prioritized bottleneck for building efficient, scalable multi-agent systems.

Our claim targets MAS in which agents read or modify shared mutable state during long inference windows, including shared repositories, blackboard memories, message buffers, and embodied world states. Systems with disjoint inputs face lower concurrency risk and may be bottlenecked by reasoning, planning, or communication quality. This scope complements MAS surveys that organize workflows, infrastructure, and collaboration patterns (Chen et al., 2024b; Li et al., 2024b; Tran et al., 2025; Guo et al., 2024); our contribution is the concurrency-control lens that connects independently developed coordination mechanisms.

Table 1. Recent evidence that concurrency mechanisms affect MAS outcomes.

Source	Concurrency signal	Reported effect
CAID	worktree isolation, merge validation	63.3% isolated vs. 55.5% unisolated; single-agent 57.2%
CodeR	dependency scheduling	22% vs. 10% resolved after removing the task graph
Silo-Bench	barriers, state conflicts	67.1% of failures; RCC reaches 100% at high contention
MegaAgent	parallel scheduling	800s vs. 4505s without parallel group execution
SagaLLM	saga transactions	correct reactive planning where baseline LLM planners fail

Table 2. Concurrency-attributable failures are a substantial fraction.

Failure Mode	Source	Rate	Concurrency Root
Premature sub-mission	Silo-Bench	37.2%	Missing sync barriers
Consensus failure	Silo-Bench	29.9%	Concurrent conflicting states
Inter-agent mis-alignment	MAST	36.9%	Stale reads, inconsistent state
Coordination overhead	Silo-Bench	RCC $\leq$ 100%	Concurrency scaling penalty

To illustrate this perspective concretely, consider a motivating example involving two coding agents collaboratively developing software on a shared file system. Agent *A* reads an existing utility module `utils.py` and subsequently implements a new feature in `main.py` that imports and

invokes a function `f_A` from the utility module. Concurrently, Agent *B* is refactoring `utils.py`, renaming `f_A` to `func_A`. Unfortunately, between the moment *A* reads the utility module and the moment *A* completes its implementation, *B* writes the refactored version. The result: both agents complete their individual tasks correctly from their own perspectives, yet the system enters an inconsistent state where `main.py` contains broken imports. This failure superficially appears to be a coordination or communication problem. However, examining it through the lens of concurrent systems reveals a classic concurrency hazard: a *stale read* leading to an inconsistent state.

What makes such hazards particularly prevalent in MAS is a fundamental *temporal asymmetry*: LLM inference time (e.g., the duration of the “thinking” phase) is typically orders of magnitude longer than the execution time of tool actions, dramatically expanding the window during which interleaved operations can produce conflicts. While an agent reasons for seconds or even minutes, other agents may modify the shared environment multiple times, invalidating the assumptions underlying the first agent’s decisions.

The remainder of this paper is organized as follows. We first demonstrate that common MAS failure modes can be systematically understood as **concurrency hazards** (§2), revealing that diverse “coordination problems” share a common structure rooted in concurrent access to shared state. We then present **recommendations** for concurrency control in MAS design (§3, Table 3), addressing objectives, system-level mechanisms, and trade-offs among correctness, efficiency, and scalability. Finally, we discuss open challenges and issue a **call to action** (§5) for cross-disciplinary collaboration between ML and systems-oriented communities.

2. Concurrency Hazards in Disguise

Many failures in multi-agent systems, often framed as coordination or communication issues, can be understood as classic concurrency hazards. We present representative failure cases and map them to established concurrency control concepts, showing that these challenges mirror problems long studied in systems and database research (Bernstein & Goodman, 1981; Bernstein et al., 1987; Weikum & Vossen, 2002). This perspective motivates treating concurrency control as a core design principle for MAS and grounding coordination heuristics in explicit concurrency semantics.

2.1. Failure Examples

We present four representative failure scenarios in multi-agent systems that are often attributed to poor coordination or communication, but are more precisely explained as classic concurrency hazards over shared state.

Stale Read. In a collaborative coding MAS (Qian et al.,

Table 3. Design space for MAS concurrency control. Trade-offs evaluated against: task success (S), compatibility (C), efficiency (E), inference cost (I). Arrows: $\uparrow$ improves, $\downarrow$ degrades.

Layer	Decision	Options (Trade-offs)
System Design	Isolation level	Weak/Read Committed (E $\uparrow$, S $\downarrow$: more parallelism, risks anomalies) $\leftrightarrow$ Strong/Serializable (S $\uparrow$, E $\downarrow$)
	Control strategy	Pessimistic/locking (S $\uparrow$, E $\downarrow$: blocks during long inference) vs. Optimistic/validation (E $\uparrow$, I $\downarrow$: wastes compute on abort)
	Versioning	Single-version (simple) vs. MVCC (E $\uparrow$: readers never block writers; C $\downarrow$: added complexity)
	Transaction granularity	Fine-grained/single action (E $\uparrow$, I $\downarrow$: shorter conflicts, higher overhead) vs. Coarse/subtask (I $\uparrow$, S $\downarrow$: expensive rollbacks)
	Transaction boundaries	Explicit BEGIN/COMMIT (C $\uparrow$, I $\downarrow$: flexible, requires model understanding) vs. Implicit/system-inferred (I $\uparrow$, C $\downarrow$)
	Lock/resource granularity	Coarse/files (I $\uparrow$, E $\downarrow$) vs. Fine/functions (E $\uparrow$, I $\downarrow$: more parallelism, more metadata)
Infrastructure	Backend system	Custom (C $\uparrow$: tailored semantics) vs. Existing DB/Git/FS (S $\uparrow$, C $\uparrow$: mature guarantees, may not fit agent semantics)
	Version control integration	Branch-per-subtask (S $\uparrow$: isolation; E $\downarrow$: merge overhead) vs. Validation-at-merge (E $\uparrow$, S $\downarrow$: deferred conflict detection)
	Inference optimization	Standard vs. Optimized batching/speculation/quantization (E $\uparrow$, S $\uparrow$: shorter transactions reduce conflict window)
	Checkpointing	None (simple) vs. KV-cache checkpointing (I $\uparrow$: efficient rollback without full recomputation; C $\downarrow$: engine support required)
Model	Concurrency training	None vs. SFT/RL on conflict scenarios (S $\uparrow$: better anticipation/resolution; I $\downarrow$: requires data and compute)
	Prompt intervention	Generic vs. Concurrency-aware prompts (I $\uparrow$: low cost; S $\pm$: limited, brittle guarantees)
	Task decomposition	Overlapping resources (E $\uparrow$, S $\downarrow$) vs. Disjoint partitioning (S $\uparrow$, C $\downarrow$: requires upfront design effort)
	Failure feedback	Opaque “retry” (I $\uparrow$: simple) vs. Semantic conflict details (S $\uparrow$, I $\downarrow$: enables adaptation, requires model capability)

2024; Hong et al., 2024), Agent *A* reads a configuration file `config.yaml` and implements a client module based on the observed endpoints. Concurrently, Agent *B* updates the configuration to reflect new infrastructure. When *A* later writes its code, it relies on outdated assumptions, resulting in an inconsistent system state despite both agents acting correctly in isolation. This is a standard stale-read anomaly.

Lost Update. Two agents independently modify different parts of a shared file `utils.py`. Agent *A* optimizes one function, while Agent *B* fixes a bug in another. Because both read the same initial version and write back independently, the later write overwrites the earlier one, silently discarding one agent’s contribution. This lost update occurs without either agent detecting a conflict.

Stale Correction. In message-based MAS (Li et al., 2023; Wu et al., 2023), Agent *A* broadcasts a plan, Agent *B* sends a correction, but Agent *C* begins executing the original plan before receiving the update. Although the correction is sent, it is not applied atomically with respect to all agents, leading to incorrect execution driven by outdated information.

Action-Message Desynchronization. In embodied environments such as Minecraft-based benchmarks (Fan et al., 2022; Wang et al., 2024a; Dong et al., 2024), agents coordinate via both messages and world-altering actions. An agent may act on a message describing an intended state that has already been invalidated by another agent’s concurrent ac-

tion. As a result, agent beliefs and the true environment state diverge, even when communication is logically consistent.

Implicit Assumptions in Existing Systems. These hazards are present in current MAS architectures. Systems such as MAGIS (Tao et al., 2024) improve reliability through structured orchestration and post-hoc Git-based merging, but implicitly assume benign interleavings during concurrent reasoning. Conflicts are detected only after agents have already performed expensive inference on incompatible assumptions. This reflects a broader pattern across MAS frameworks that rely on orchestration while leaving concurrency semantics underspecified. Recent systems show the same pattern from another angle. CAID uses isolated worktrees and merge-time validation, CodeR uses a task graph to order dependent work, SagaLLM uses saga-style compensation, and MegaAgent emphasizes parallel scheduling (Geng & Neubig, 2026; Chen et al., 2024a; Chang & Geng, 2025; Wang et al., 2024b). These mechanisms correspond to optimistic isolation, dependency scheduling, transactional recovery, and concurrent scheduling, even when they are introduced without explicit concurrency-control terminology.

2.2. A Unified Framework

The failure examples above, though arising in different application contexts, share a common structure. We now develop

a unified framework that reveals these failures as violations of fundamental concurrency properties, drawing on classical concurrency control theory (Bernstein et al., 1987; Weikum & Vossen, 2002; Adya, 1999).

Formalizing Multi-Agent Systems. We model a multi-agent system as a collection of n agents with respective internal states $\{\mathcal{A}^1, \mathcal{A}^2, \dots, \mathcal{A}^n\}$, all operating within a *shared* environment $\mathcal{E}$. Each agent j executes a sequence of actions $a_1^j, a_2^j, \dots$ according to its own local ordering, but these sequences interleave concurrently across agents, i.e., actions may overlap temporally, and conflicting operations on shared resources can occur.

We classify actions based on whether they modify the shared environment. An action is *side-effect-free* (a *read*) if it leaves $\mathcal{E}$ unchanged; examples include reading a file, observing game state, or retrieving messages. An action is *side-effecting* (a *write*) if it modifies $\mathcal{E}$; examples include writing a file, placing a game object, or broadcasting a message.

We further distinguish between *agent-local state* and *environment state* based on *mutability by other agents*: environment state can be modified by any agent’s writes, while agent-local state (e.g., context window, reasoning trace) can only be directly modified by that agent itself. In coding MAS, the shared file system constitutes environment state. In message-passing MAS, each agent’s mailbox, which is writable by other agents via message sends, is environment state. In game environments, both world state and communication channels are environment state.

A key observation is that LLM-based reasoning is side-effect-free with respect to $\mathcal{E}$, but requires substantial time to complete. This *temporal asymmetry*, i.e., inference spans seconds to minutes while tool execution completes in milliseconds, dramatically expands the window during which interleaved operations can produce conflicts.

Consistency and Isolation. We identify two properties whose violation underlies MAS failures, drawing on the ACID properties from database theory (Härder & Reuter, 1983):

Consistency requires that concurrent operations do not leave the system in a conflicting or contradictory state. In MAS, this means the combined effects of all agents’ actions should be mutually compatible: two agents should not simultaneously assume exclusive use of the same resource, and actions taken based on observations should remain valid when executed. Consistency is a *global* property concerning system-wide coherence.

Isolation requires that each agent’s execution proceeds “as if” it were the only agent in the system. Formally, the outcome should be equivalent to some serial execution of all

agents’ actions (Weikum & Vossen, 2002). Isolation is an *agent-centric* property: no agent should observe another agent’s incomplete operation sequence. The classical notion of *serializability*, that concurrent execution should be equivalent to some sequential ordering, captures the strongest form of isolation (Papadimitriou, 1979).

These properties are complementary: consistency ensures global coherence, while isolation ensures local predictability. Violations of either lead to the anomalies described above.

Mapping Failures to the Framework. In *coding MAS*, the *write-after-read* failure violates both consistency (Agent A ’s implementation conflicts with Agent B ’s configuration changes) and isolation (Agent A ’s read was invalidated by Agent B ’s concurrent write). The *lost update* failure similarly violates both: consistency is violated because only one agent’s modifications persist, and isolation is violated because one agent’s write was silently overwritten.

In *message-based MAS*, the *stale correction* failure maps directly to the write-after-read pattern: Agent C reads the original plan, Agent B writes the correction, but Agent C ’s actions proceed based on the stale read. In *cooperative game MAS*, the *action-message desynchronization* failure violates consistency because the coordination message and world state become inconsistent, and isolation because Agent B ’s operation is disrupted by Agent C ’s uncoordinated action.

Implications for MAS Design. The unified framework identifies failures documented across diverse MAS applications as systematic consequences of uncontrolled concurrency. The database and distributed systems communities have developed extensive techniques to address precisely these problems: locking protocols (Eswaran et al., 1976), optimistic concurrency control (Kung & Robinson, 1981), multi-version concurrency control (Reed, 1978), and various isolation levels (Berenson et al., 1995; Adya, 1999). For MAS research, the central task is adapting classical concurrency control mechanisms to the unique characteristics of LLM-based agents: long inference latencies, natural-language interfaces, and imperfect adherence to specifications. We address this task in the following section.

3. Concurrency Control for MAS

Having established that many MAS failures stem from concurrency hazards, we now address the central question: how should MAS designers approach concurrency control? This section articulates design objectives and examines the mechanisms available to system architects, organized across three complementary layers (Table 3): system design, infrastructure and model capabilities. Together, these layers provide a comprehensive framework for understanding where concurrency control responsibilities reside and how they may be systematically addressed.

3.1. Design Objectives

Effective concurrency control for MAS should balance multiple, often competing objectives. We identify four primary dimensions that collectively define the quality of a concurrency control solution.

Task Success Rate. The fundamental objective is ensuring that concurrent agent execution produces correct results. What constitutes “success” depends on the application: in coding tasks, it may mean passing all tests without introducing bugs; in collaborative writing, it may mean producing coherent output without contradictory edits. A concurrency control mechanism should maximize task success rate across diverse MAS workloads. Existing MAS benchmarks (Liu et al., 2024b; Cemri et al., 2025; Jimenez et al., 2024; Zhou et al., 2024) provide starting points for measuring success, though they largely ignore concurrency-specific failure modes.

Compatibility. Concurrency control mechanisms should accommodate diverse MAS architectures: centralized orchestrators (Wu et al., 2023), decentralized peer-to-peer designs (Li et al., 2023), hybrid topologies (Hong et al., 2024), and hierarchical structures (Qian et al., 2024). A mechanism tightly coupled to a specific communication pattern or agent architecture limits reusability. Ideally, concurrency control primitives should be orthogonal to the MAS coordination strategy, enabling modular integration without redesigning the entire system.

Execution Efficiency. Concurrency control inherently restricts parallelism to ensure correctness, creating tension with performance goals. Key efficiency metrics include: (i) *end-to-end task completion time*: the wall-clock duration for the MAS to finish a task; (ii) *effective parallelism*: the degree to which agents operate concurrently without blocking; and (iii) *abort/retry rate*: the frequency of transaction rollbacks due to detected conflicts. An ideal mechanism minimizes completion time while maintaining correctness guarantees.

Inference Cost. MAS concurrency control incurs costs beyond those of traditional distributed systems: direct overhead from tool calls for lock acquisition or conflict detection; model comprehension cost, as LLMs may struggle with concurrency primitives and require extended prompts or fine-tuning; and context window consumption from concurrency-related instructions and state tracking. Critically, mechanisms requiring agents to explicitly manage locks or reason about conflicts demand far greater model sophistication than transparent system-level enforcement.

3.2. System Design for MAS

We now examine the theoretical foundations and concrete design choices for MAS concurrency control, spanning iso-

lation levels, control strategies, primitives, and integration with broader MAS mechanisms.

3.2.1. THEORETICAL FOUNDATIONS

Isolation Levels. Database systems formalize correctness through *isolation levels*, which specify the degree to which transactions are protected from interference (Berenson et al., 1995; Adya, 1999). The classical hierarchy includes, from weakest to strongest: Read Uncommitted, Read Committed (RC), Repeatable Read (RR), Snapshot Isolation (SI), Serializable (SER), and Strict Serializable (SSER) (Bernstein et al., 1987; Fekete et al., 2005; Papadimitriou, 1979). Each level prevents progressively more anomalies (e.g., dirty reads, non-repeatable reads, phantom reads, and write skew) but typically incurs higher overhead (Cahill et al., 2009).

For MAS, the appropriate isolation level depends critically on the application domain. A multi-agent coding system modifying a shared codebase may require serializability to prevent silent corruption from conflicting edits. In contrast, a multi-agent debate system where agents operate on largely disjoint logical resources (their own arguments) may tolerate weaker guarantees like Read Committed, trading strict isolation for higher throughput. The general idea is that stronger isolation provides better guarantees to agents but may reduce performance and increase implementation complexity, which is a tradeoff that should be calibrated per application.

Pessimistic vs. Optimistic Control. Concurrency control strategies fall into two broad categories (Bernstein & Goodman, 1981; Kung & Robinson, 1981). *Pessimistic* approaches, such as two-phase locking (2PL), acquire locks before accessing resources, blocking other agents until locks are released (Eswaran et al., 1976; Gray et al., 1976). This prevents conflicts proactively but may cause deadlocks (circular waits) and reduces parallelism. *Optimistic* approaches allow agents to proceed without locks, validating at commit time that no conflicts occurred; conflicting transactions abort and retry. Optimistic methods suit low-contention workloads but may thrash under high contention.

A crucial distinction from traditional systems is the *temporal asymmetry* in MAS: LLM inference typically spans seconds to minutes, while simple tool calls complete in milliseconds. This asymmetry fundamentally alters the calculus. Under pessimistic control, a lock held during LLM reasoning blocks other agents for extended periods, severely degrading parallelism. Under optimistic control, aborts waste substantial compute: an agent may reason for minutes only to have its transaction invalidated. Neither approach dominates; the choice depends on contention levels, task structure, and the relative costs of blocking versus retry. Recent coding-agent evidence illustrates this trade-off: CAID reports that isolated branches with merge-time validation

outperform a shared-workspace multi-agent baseline, suggesting that optimistic isolation is effective when conflicts can be validated cheaply at integration time (Geng & Neubig, 2026).

Multiversion Concurrency Control. Multiversion concurrency control (MVCC) offers a middle ground by maintaining multiple versions of data (Reed, 1978; Bernstein et al., 1987). Readers access consistent snapshots without blocking writers, enabling the principle that “readers never block writers and writers never block readers.” MVCC naturally supports Snapshot Isolation (Fekete et al., 2005) and forms the basis for concurrency control in many modern databases. For MAS, MVCC-style approaches could allow agents to observe consistent environment states while others make modifications, with conflict detection deferred to commit time. The challenge lies in defining “versions” for arbitrary agent environments beyond traditional databases.

3.2.2. PRIMITIVE DESIGN

Transaction Granularity. A core design decision is the unit of atomicity. Transactions may range from individual actions (e.g., a single tool call) to entire agent subtasks (e.g., “implement feature X”). Fine-grained transactions shorten conflict windows but incur higher coordination overhead, while coarse-grained transactions reduce overhead at the cost of higher conflict probability and more expensive rollbacks.

Another question is whether transaction boundaries are *explicit*, via primitives such as `BEGIN/COMMIT`, or *implicit*, inferred by the system from agent behavior or task structure. Explicit control gives agents flexibility but assumes that LLMs can reliably reason about transactional semantics. Implicit control lowers cognitive burden but risks poorly chosen boundaries that either fragment logical work or bundle too much state into a single transaction.

Resource Abstraction and Lock Types. Lock-based designs should define what constitutes a lockable resource: entire files, individual functions, database rows, or higher-level semantic objects (e.g., “a calendar slot”). A *fine-grained resource* is the smallest unit to which access control applies. In a codebase, file-level locking treats `utils.py` as one resource, while function-level locking treats `parse_config` and `validate_input` as distinct resources. Function-level locking allows two agents to modify different functions concurrently, whereas file-level locking serializes them unnecessarily. This mirrors row-level and table-level locking in databases (Gray et al., 1976). Finer-grained resources enable more parallelism but require more complex tracking and metadata.

Lock modes further regulate access (Gray et al., 1976). *Shared (read) locks* permit concurrent observation, while *ex-*

clusive (write) locks serialize modifications. This distinction aligns well with MAS workloads, where reads are frequent and typically safe to overlap, whereas writes should be carefully coordinated. More expressive mechanisms, such as intention or predicate locks, may be useful for hierarchical or semantic resources.

Conflict Detection and Rollback. Optimistic control requires detecting conflicts by tracking *read sets* and *write sets*, then validating that no incompatible updates occurred concurrently (Kung & Robinson, 1981; Adya, 1999). Software transactional memory (STM) offers a reference model for such tracking (Shavit & Touitou, 1995; Harris et al., 2005), though MAS environments are more heterogeneous than the shared-memory settings STM targets. This heterogeneity is an analytical distinction: MAS resources extend beyond memory locations to files, prompts, tool outputs, mailboxes, external APIs, and natural-language commitments. Beyond conflict detection within the MAS runtime itself, recent black-box and history-based database isolation checkers suggest a complementary direction: treating MAS executions as observable histories and using an external monitor to check whether interactions satisfy different consistency or isolation levels (Tan et al., 2020; Liu et al., 2024a; Cai et al., 2025).

When conflicts arise, aborted transactions should be rolled back. In MAS, rollback is challenging because agent state includes external writes, internal context, and intermediate reasoning. Techniques such as KV-cache checkpointing (Kwon et al., 2023) may allow partial rewinding without full re-inference. However, actions with irreversible side effects (e.g., sending emails or invoking external APIs) fundamentally limit rollback and should be treated with special care.

3.2.3. MAS INTEGRATION

Synchronization Granularity. MAS frameworks should decide on the *minimal synchronization unit*: the finest granularity at which agents coordinate. One approach uses *round-based synchronization*: agents act in discrete rounds, with a barrier ensuring all agents complete round k before any begins round $k + 1$. This simplifies reasoning about concurrent state but limits parallelism and may be inefficient when agents’ tasks have heterogeneous durations. The alternative is *fully asynchronous* execution where agents proceed independently, relying solely on concurrency control primitives (locks, validation) for coordination. This maximizes parallelism but requires more sophisticated conflict handling.

The synchronization granularity should align with transaction boundaries. If transactions span multiple rounds in a round-based system, mid-transaction preemption creates complex partial states. Conversely, if transactions complete within single rounds, the round barrier naturally provides

commit points.

Inference Interruption. A distinctive aspect of MAS is whether ongoing inference can be interrupted. An agent mid-reasoning may be holding logical locks or accumulating reads that will be validated at commit. If another agent’s commit invalidates these reads, should the first agent be *preempted* (immediately aborted) or allowed to complete then fail validation? Preemption avoids wasted compute but requires mechanisms to safely interrupt LLM inference and restore state. Allowing completion then validation is simpler but wastes resources when conflicts are predictable.

Related questions include whether agents can *voluntarily* yield execution (cooperative scheduling) and how interrupted agents resume from the beginning of the transaction, from a checkpoint, or with partial context preserved via KV cache.

Semantic Feedback on Failures. When a transaction fails due to conflict detection, lock timeout, or validation failure, the agent receives an error signal. A key design choice is whether to provide *semantic explanations* of failures. A rich explanation might state: “your write to `utils.py` conflicted with Agent B’s concurrent modification of the `parse_config` function.” An opaque signal simply indicates “transaction aborted, please retry.”

Semantic feedback enables agents to reason about conflicts and adapt strategies—perhaps avoiding the contested resource, coordinating with the conflicting agent, or decomposing the task differently. However, this requires models capable of interpreting and acting on such information, a capability that may require specific training or careful prompting. Opaque signals are simpler but provide no guidance for conflict resolution.

Integration with Existing Pipelines. A practical question is how to augment existing MAS workflows, which typically assume benign interleavings, with explicit concurrency semantics. Consider systems like MAGIS (Tao et al., 2024) that use structured role specialization and Git-based collaboration. Several “drop-in” strategies could add concurrency guarantees without redesigning the entire pipeline:

- *Branch-per-subtask*: Each agent subtask operates on an isolated Git branch, with merges gated by automated conflict detection and semantic validation (e.g., test execution, type checking).
- *Transactional file edits*: Wrap multi-file modifications in lightweight transactions that abort if concurrent changes to the same files are detected before commit.
- *Validation-at-merge*: Defer validation to merge time and enrich merge checks with semantic constraints beyond textual diff resolution.

These strategies trade off eager conflict prevention against

deferred conflict detection. Branch-per-subtask maximizes isolation but complicates cross-branch dependencies; validation-at-merge preserves existing workflows but may waste computation on branches that ultimately conflict.

3.3. Infrastructure Support

MAS designers can leverage existing infrastructure that already provides well-tested concurrency primitives.

File Systems and Databases. File systems offer atomic operations and advisory locking, while relational databases provide full ACID transactions with configurable isolation levels (Härder & Reuter, 1983; Weikum & Vossen, 2002). Vector databases increasingly expose transactional semantics as well (Pinecone Systems, 2023; Weaviate, 2023). Structuring agent interactions around such systems allows MAS to inherit mature concurrency guarantees and performance optimizations, offloading much of the correctness burden.

Version Control Systems. Version control systems such as Git support concurrent work via branching and explicit merge-based conflict resolution. Recent work explores Git as coordination infrastructure for MAS. EvoGit (Huang et al., 2025) uses Git’s DAG structure to enable asynchronous multi-agent development, isolating agents on branches and synchronizing via merges while avoiding global serialization. This design favors scalability and fault isolation, but defers semantic conflict resolution to merge time, where textual diffs may not capture program-level invariants.

Context Management Infrastructure. Git-like abstractions have also been applied to agent context management. GCC (Wu, 2025) provides branching and merging for agent memory, enabling efficient context reuse and performance gains. However, it targets primarily single-agent settings and does not define concurrency semantics for multi-agent interleavings over shared context.

Across these systems, a common pattern emerges: infrastructure supplies powerful *mechanisms* (transactions, branches, merges), while higher-level frameworks must still define *policies*: when to isolate, what conflicts are acceptable, and how to enforce semantic invariants. Bridging this gap between infrastructure capabilities and concurrency semantics is central to the agenda we advocate.

Inference Engine Efficiency. The temporal asymmetry that exacerbates MAS concurrency issues (i.e., long inference latencies relative to tool execution) can be mitigated by faster inference. Advances including continuous batching (Yu et al., 2022b), paged attention (Kwon et al., 2023), speculative decoding (Leviathan et al., 2023), and quantization (Frantar et al., 2022) reduce per-token latency and thus shrink the window during which an agent holds logi-

cal locks or accumulates stale reads. From a concurrency control perspective, faster inference is equivalent to shorter transactions, directly reducing conflict probability.

Efficient Checkpointing for Rollback. Optimistic concurrency control requires efficient rollback. In LLM inference, this translates to restoring the KV cache to a prior state. Modern inference engines with paged KV cache management (Kwon et al., 2023) can potentially support *forking* (copying cache state for speculative execution) and *rollback* (discarding cache entries added after a checkpoint). Explicit support for these operations would enable efficient transaction abort without full recomputation. Cache sharing across agents with common prefixes (Zheng et al., 2024) further reduces the cost of retry by amortizing prompt processing.

3.4. Model Capabilities

Complementing system-level approaches, the underlying LLMs can be improved to exhibit concurrency-aware behavior through evaluation, training, and inference-time interventions.

3.4.1. BENCHMARKING CONCURRENCY AWARENESS

A prerequisite for improving model capabilities is measuring them. We advocate for benchmarks that evaluate: (i) an agent’s ability to *anticipate* conflicts given knowledge of other agents’ goals and current activities; (ii) an agent’s ability to *detect* conflicts from environmental feedback (e.g., failed writes, stale reads); and (iii) an agent’s ability to *resolve* conflicts through negotiation, backing off, or requesting arbitration. Such benchmarks could extend existing MAS evaluation suites (Liu et al., 2024b; Cemri et al., 2025) with concurrency-specific metrics: conflict rate, resolution success rate, wasted computation from aborts, and throughput under contention.

A particularly valuable benchmark type would measure agent behavior *without* explicit concurrency control, revealing how naturally (or poorly) current models handle concurrent settings and what failure modes emerge most frequently.

3.4.2. TRAINING FOR CONCURRENCY AWARENESS

Given appropriate benchmarks and environments, LLMs can be trained to exhibit concurrency-aware behavior. Supervised fine-tuning (SFT) on traces of successful conflict anticipation, detection, and resolution can instill basic patterns. Reinforcement learning (RL) in multi-agent environments, building on the substantial MARL literature (Lowe et al., 2017; Foerster et al., 2018; Yu et al., 2022a; Zhang et al., 2019), can train agents to coordinate under contention, learning when to yield, retry, or escalate.

The reward signal should reflect both task success and co-

ordination efficiency, penalizing unnecessary conflicts and wasted retries. Centralized training with decentralized execution can mitigate non-stationarity when agents share rewards and training infrastructure. The harder case is heterogeneous deployment, where agents from different providers or training regimes coordinate without prior joint training, a setting closely related to Ad Hoc Teamwork (Stone et al., 2010; Barrett & Stone, 2015). In such deployments, as one agent’s policy or system prompt changes, the effective environment for other agents shifts, complicating convergence and reliable coordination (Hernandez-Leal et al., 2017). Techniques from cooperative MARL, including centralized training with decentralized execution and opponent modeling, may help address this challenge.

3.4.3. INFERENCE-TIME INTERVENTIONS

Without modifying models, some concurrency-aware behavior can be induced through prompt design and task structuring.

Prompt Engineering. Agents can be explicitly informed that they operate in a concurrent environment with shared resources and guided toward conservative behaviors, such as checking for concurrent edits, acquiring resources in a consistent order, or retrying after failed writes. Such prompts rely on the base model’s instruction-following ability and whatever concurrency knowledge it has implicitly acquired during pretraining, and thus offer limited and brittle guarantees.

Task Decomposition and Orchestration. Contention can often be reduced by decomposing tasks so that agents operate on largely disjoint resources, with hierarchical approaches dividing responsibilities between strategic and tactical agents to reduce coordination complexity. While perfect partitioning is rarely possible, careful decomposition can substantially lower conflict frequency. These approaches trade upfront design effort for reduced runtime conflicts, but scale poorly to novel or highly interdependent workloads where explicit concurrency control becomes necessary.

4. Alternative Views

Foundation model capabilities alone suffice. One view holds that sufficiently capable models can resolve conflicts without explicit concurrency control. Conversation-centric frameworks such as AutoGen exemplify the appeal of relying on model capability and dialogue structure for coordination (Wu et al., 2023). However, as agent count grows, conflict probability approaches certainty regardless of model quality. Natural-language negotiation wastes inference budget, coordination embedded in model weights is neither debuggable nor auditable, and robust concurrency aware-

ness would require training data that does not exist. Model behavior also remains non-deterministic in practice (Atil et al., 2024; Kahng et al., 2024).

Traditional systems techniques suffice. Another view argues that existing database transactions and locks can be directly applied to MAS. However, LLM agents are semantic-driven, non-deterministic (Atil et al., 2024), and incur inference costs orders of magnitude larger than system-level operations. These mismatches make direct transplantation ineffective and instead call for system–model co-design.

Eventual consistency and convergence suffice. A third view suggests that eventual consistency (e.g., via CRDTs) is sufficient (Shapiro et al., 2011; Kleppmann & Beresford, 2017). This conflates convergence with semantic correctness. While replicas may eventually agree, application invariants can be violated when agents reason over long inference windows based on invalidated state. CRDTs suit independent operations with well-defined merge semantics, but MAS often rely on cross-resource invariants that should remain stable during reasoning. The key question is what guarantees agents can rely on while they think and act.

Existing community tools already cover the issue. Existing tool (SE branching and CI, MARL coordination, and distributed consistency protocol) are valuable starting points, but LLM-MAS uniquely combines shared-state hazards with long, costly, language-mediated inference. The open problem is exposing these as agent-facing primitives with clear isolation, validation, and rollback semantics.

5. Call to Action

Many coordination failures in multi-agent systems are, at their core, concurrency control problems. The anomalies well-known from parallel programs and distributed databases resurface when agents share mutable state, amplified by long LLM inference windows and non-determinism. Closing this gap requires cross-disciplinary collaboration. ML researchers should treat concurrency as a first-class systems concern rather than emergent behavior to be prompted around, while researchers in PL and DB should adapt classical techniques to LLM-based agents through careful redesign. Neither perspective alone is sufficient.

5.1. For ML Researchers

Recognize concurrency as a distinct failure mode. When MAS exhibit timing-dependent errors or coordination breakdowns despite locally correct actions, the root cause is often concurrent access to shared state under long inference windows. Treating these as generic coordination problems obscures the underlying issue. Explicit recognition is the first step toward systematic solutions.

Design benchmarks with contention in mind. Current evaluations (Liu et al., 2024b; Jimenez et al., 2024; Zhou et al., 2024) report aggregate success rates but cannot distinguish capability failures from concurrency failures. A system achieving 70% success may fail 20% of the time due to stale reads alone, yet this remains invisible in current metrics. New benchmarks should vary contention levels systematically and measure conflict frequency, resolution success, wasted computation from aborts, and effective parallelism under contention.

Train for concurrency awareness. Pretraining corpora contain extensive coverage of sequential programming but minimal exposure to concurrent coordination patterns such as lock denial, abort recovery, or conflict anticipation under partial observability. Whether models can acquire robust concurrency-aware behavior via prompting alone or require explicit training with multi-agent RL (Lowe et al., 2017; Foerster et al., 2018) remains an open and consequential question.

5.2. For Researchers in Systems-Oriented Fields

Adapt, not transplant. Databases and distributed systems offer mature foundations: serializability theory (Papadimitriou, 1979), isolation hierarchies (Berenson et al., 1995; Adya, 1999), and protocols such as 2PL (Eswaran et al., 1976), OCC (Kung & Robinson, 1981), MVCC (Reed, 1978), and SSI (Cahill et al., 2009). However, LLM agents interact through natural language and are non-deterministic, making direct transplantation ineffective. The core challenge is determining which abstractions transfer, which require redesign, and what new primitives are needed.

Design for latency asymmetry. Database transactions complete in microseconds; LLM inference takes seconds to minutes. Pessimistic locking blocks unacceptably at these timescales; optimistic control wastes expensive computation on aborts. New protocols should explicitly account for this asymmetry, potentially via interruptible inference, KV-cache rollback, or contention-aware scheduling. The success criterion is preserving task success while improving effective parallelism and reducing wasted tokens under controlled contention.

Build agent-friendly concurrency infrastructure. MAS currently lack the primitives that make concurrency manageable in traditional systems: versioning, conflict detection, and rollback semantics exposed through interfaces that agents can reliably use. Equally important are observability and debugging tools that make concurrency failures explicit and actionable. Without such tooling, diagnosing whether a failure originates from a race condition, a coordination misalignment, or a model-level error remains prohibitively difficult in practice. Building this infrastructure is a prerequisite for principled progress.

Acknowledgements

We thank Dr. Si Liu at ETH Zürich for constructive discussions on concurrency control topics.

Limitations

While this position paper establishes a compelling conceptual mapping between MAS failures and classical concurrency anomalies, our framework primarily targets systems with explicitly shared mutable state, and may not generalize to MAS architectures that rely on implicit coordination or emergent communication. Additionally, the trade-offs among correctness, efficiency, and inference cost are discussed qualitatively; more rigorous empirical validation across diverse real-world workloads is left for future work.

References

- Adya, A. Weak consistency: A generalized theory and optimistic implementations for distributed transactions. Technical report, Massachusetts Institute of Technology, USA, 1999.
- Anonymous. Position: Science is collaborative—Illm for science should be too. Concurrent Submission to ICML, 2026. Filename in supplementary material: 52.pdf.
- Atil, B., Aykent, S., Chittams, A., Fu, L., Passonneau, R. J., Radcliffe, E., Rajagopal, G. R., Sloan, A., Tudrej, T., Ture, F., Wu, Z., Xu, L., and Baldwin, B. Non-determinism of “deterministic” llm settings, 2024. URL <https://arxiv.org/abs/2408.04667>.
- Barrett, S. and Stone, P. Cooperating with unknown teammates in complex domains: A robot soccer case study of ad hoc teamwork. In Bonet, B. and Koenig, S. (eds.), *Proceedings of the Twenty-Ninth AAAI Conference on Artificial Intelligence, January 25-30, 2015, Austin, Texas, USA*, pp. 2010–2016. AAAI Press, 2015. doi: 10.1609/AAAI.V29I1.9428. URL <https://doi.org/10.1609/aaai.v29i1.9428>.
- Berenson, H., Bernstein, P. A., Gray, J., Melton, J., O’Neil, E. J., and O’Neil, P. E. A critique of ANSI SQL isolation levels. In Carey, M. J. and Schneider, D. A. (eds.), *Proceedings of the 1995 ACM SIGMOD International Conference on Management of Data, San Jose, California, USA, May 22-25, 1995*, pp. 1–10. ACM Press, 1995. doi: 10.1145/223784.223785. URL <https://doi.org/10.1145/223784.223785>.
- Bernstein, P. A. and Goodman, N. Concurrency control in distributed database systems. *ACM Comput. Surv.*, 13(2):185–221, 1981. doi: 10.1145/356842.356846. URL <https://doi.org/10.1145/356842.356846>.
- Bernstein, P. A., Hadzilacos, V., and Goodman, N. *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987. ISBN 0-201-10715-5. URL <http://research.microsoft.com/en-us/people/philbe/ccontrol.aspx>.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. In Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., and Lin, H. (eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- Cahill, M. J., Röhm, U., and Fekete, A. D. Serializable isolation for snapshot databases. *ACM Trans. Database Syst.*, 34(4):20:1–20:42, 2009. doi: 10.1145/1620585.1620587. URL <https://doi.org/10.1145/1620585.1620587>.
- Cai, Z., Liu, S., Wei, H., Chen, Y., and Pan, A. Fast verification of strong database isolation. *Proc. VLDB Endow.*, 19(4):563–575, 2025. URL <https://www.vldb.org/pvldb/vol19/p563-wei.pdf>.
- Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A. G., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., and Stoica, I. Why do multi-agent LLM systems fail? *CoRR*, abs/2503.13657, 2025. doi: 10.48550/ARXIV.2503.13657. URL <https://doi.org/10.48550/arXiv.2503.13657>.
- Chang, E. Y. and Geng, L. Sagallm: Context management, validation, and transaction guarantees for multi-agent LLM planning. *Proc. VLDB Endow.*, 18(12):4874–4886, 2025. doi: 10.14778/3750601.3750611. URL <https://www.vldb.org/pvldb/vol18/p4874-chang.pdf>.
- Chen, D., Lin, S., Zeng, M., Zan, D., Wang, J., Cheshkov, A., Sun, J., Yu, H., Dong, G., Aliev, A., Wang, J., Cheng, X., Liang, G., Ma, Y., Bian, P., Xie, T., and Wang, Q. Coder: Issue resolving with multi-agent and task graphs. *CoRR*, abs/2406.01304, 2024a. doi: 10.48550/ARXIV.2406.01304. URL <https://doi.org/10.48550/arXiv.2406.01304>.

- Chen, S., Liu, Y., Han, W., Zhang, W., and Liu, T. A survey on llm-based multi-agent system: Recent advances and new frontiers in application. *arXiv preprint arXiv:2412.17481*, 2024b.
- Cheng, X., Zeng, K., Cao, Z., Dai, L., Gao, W., Han, F., Jian, A., Hong, F., Hu, W., Huang, Z., Kong, D., Leng, J., Liao, Z., Liu, P., Lin, J., Ma, X., Ruan, J., Song, J., Tan, X., Xiao, R., Yu, W., Zhan, W., Zhang, H., Zhou, C., Zhou, H., Zheng, S., Chen, R., Chen, S., Chen, Z., Dong, Y., Fan, Y., Fang, Y., Gan, Y., Guo, S., He, Q., Hu, C., Li, B., Li, D., Li, X., Li, Y., Liu, C., Liu, X., Lv, J., Ma, Q., Pan, J., Qin, C., Sun, C., Sun, W., Wang, Z., Wuerkaixi, A., Yang, X., Yuan, F., Zhu, Y., Zhai, T., Zhang, J., Zhang, R., Xu, Y., Zhao, Y., Wang, Y., Cai, X., Hu, Y., Liu, C., Pan, L., Wang, X., Xiao, B., Yao, W., Zhou, Q., and Zhu, B. Higher satisfaction, lower cost: A technical report on how llms revolutionize meituan’s intelligent interaction systems. *CoRR*, abs/2510.13291, 2025. doi: 10.48550/ARXIV.2510.13291. URL <https://doi.org/10.48550/arXiv.2510.13291>.
- Dong, Y., Zhu, X., Pan, Z., Zhu, L., and Yang, Y. Vilageragent: A graph-based multi-agent framework for coordinating complex task dependencies in minecraft. In Ku, L., Martins, A., and Srikumar, V. (eds.), *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, Findings of ACL, pp. 16290–16314. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.FINDINGS-ACL.964. URL <https://doi.org/10.18653/v1/2024.findings-acl.964>.
- Eswaran, K. P., Gray, J., Lorie, R. A., and Traiger, I. L. The notions of consistency and predicate locks in a database system. *Commun. ACM*, 19(11):624–633, 1976. doi: 10.1145/360363.360369. URL <https://doi.org/10.1145/360363.360369>.
- Fan, L., Wang, G., Jiang, Y., Mandlkar, A., Yang, Y., Zhu, H., Tang, A., Huang, D., Zhu, Y., and Anandkumar, A. Minedojo: Building open-ended embodied agents with internet-scale knowledge. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/74a67268c5cc5910f64938cac4526a90-Abstract-Datasets_and_Benchmarks.html.
- Fekete, A. D., Liarokapis, D., O’Neil, E. J., O’Neil, P. E., and Shasha, D. E. Making snapshot isolation serializable. *ACM Trans. Database Syst.*, 30(2):492–528, 2005. doi: 10.1145/1071610.1071615. URL <https://doi.org/10.1145/1071610.1071615>.
- Foerster, J. N., Chen, R. Y., Al-Shedivat, M., Whiteson, S., Abbeel, P., and Mordatch, I. Learning with opponent-learning awareness. In André, E., Koenig, S., Dastani, M., and Sukthankar, G. (eds.), *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems, AAMAS 2018, Stockholm, Sweden, July 10-15, 2018*, pp. 122–130. International Foundation for Autonomous Agents and Multiagent Systems Richland, SC, USA / ACM, 2018. URL <http://dl.acm.org/citation.cfm?id=3237408>.
- Frantar, E., Ashkboos, S., Hoeffler, T., and Alistarh, D. GPTQ: accurate post-training quantization for generative pre-trained transformers. *CoRR*, abs/2210.17323, 2022. doi: 10.48550/ARXIV.2210.17323. URL <https://doi.org/10.48550/arXiv.2210.17323>.
- Geng, J. and Neubig, G. Effective strategies for asynchronous software engineering agents. *CoRR*, abs/2603.21489, 2026. doi: 10.48550/ARXIV.2603.21489. URL <https://doi.org/10.48550/arXiv.2603.21489>.
- Gray, J., Lorie, R. A., Putzolu, G. R., and Traiger, I. L. Granularity of locks and degrees of consistency in a shared data base. In Nijssen, G. M. (ed.), *Modelling in Data Base Management Systems, Proceeding of the IFIP Working Conference on Modelling in Data Base Management Systems, Freudenstadt, Germany, January 5-8, 1976*, pp. 365–394. North-Holland, 1976.
- Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., and Zhang, X. Large language model based multi-agents: A survey of progress and challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI 2024, Jeju, South Korea, August 3-9, 2024*, pp. 8048–8057. ijcai.org, 2024. URL <https://www.ijcai.org/proceedings/2024/890>.
- Härder, T. and Reuter, A. Principles of transaction-oriented database recovery. *ACM Comput. Surv.*, 15(4):287–317, 1983. doi: 10.1145/289.291. URL <https://doi.org/10.1145/289.291>.
- Harris, T., Marlow, S., Jones, S. L. P., and Herlihy, M. Composable memory transactions. In Pingali, K., Yelick, K. A., and Grimshaw, A. S. (eds.), *Proceedings of the ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP 2005, June 15-17, 2005, Chicago, IL, USA*, pp. 48–60. ACM, 2005. doi: 10.1145/1065944.1065952. URL <https://doi.org/10.1145/1065944.1065952>.

- Hernandez-Leal, P., Kaisers, M., Baarslag, T., and de Cote, E. M. A survey of learning in multiagent environments: Dealing with non-stationarity. *CoRR*, abs/1707.09183, 2017. URL <http://arxiv.org/abs/1707.09183>.
- Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., and Schmidhuber, J. Metagpt: Meta programming for A multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=VtmBAGCN7o>.
- Huang, B., Cheng, R., and Tan, K. C. Evogit: Decentralized code evolution via git-based multi-agent collaboration. *CoRR*, abs/2506.02049, 2025. doi: 10.48550/ARXIV.2506.02049. URL <https://doi.org/10.48550/arXiv.2506.02049>.
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K. R. Swe-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=VTF8yNQm66>.
- Kahng, M., Tenney, I., Pushkarna, M., Liu, M. X., Wexler, J., Reif, E., Kallarakal, K., Chang, M., Terry, M., and Dixon, L. LLM comparator: Visual analytics for side-by-side evaluation of large language models. In Mueller, F. F., Kyburz, P., Williamson, J. R., and Sas, C. (eds.), *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems, CHI EA 2024, Honolulu, HI, USA, May 11-16, 2024*, pp. 216:1–216:7. ACM, 2024. doi: 10.1145/3613905.3650755. URL <https://doi.org/10.1145/3613905.3650755>.
- Kim, Y., Gu, K., Park, C., Park, C., Schmidgall, S., Heydari, A. A., Yan, Y., Zhang, Z., Zhuang, Y., Malhotra, M., Liang, P. P., Park, H. W., Yang, Y., Xu, X., Du, Y., Patel, S. N., Althoff, T., McDuff, D., and Liu, X. Towards a science of scaling agent systems. *CoRR*, abs/2512.08296, 2025. doi: 10.48550/ARXIV.2512.08296. URL <https://doi.org/10.48550/arXiv.2512.08296>.
- Kleppmann, M. and Beresford, A. R. A conflict-free replicated JSON datatype. *IEEE Trans. Parallel Distributed Syst.*, 28(10):2733–2746, 2017. doi: 10.1109/TPDS.2017.2697382. URL <https://doi.org/10.1109/TPDS.2017.2697382>.
- Kung, H. T. and Robinson, J. T. On optimistic methods for concurrency control. *ACM Trans. Database Syst.*, 6(2):213–226, 1981. doi: 10.1145/319566.319567. URL <https://doi.org/10.1145/319566.319567>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with pagedattention. In Flinn, J., Seltzer, M. I., Druschel, P., Kaufmann, A., and Mace, J. (eds.), *Proceedings of the 29th Symposium on Operating Systems Principles, SOSP 2023, Koblenz, Germany, October 23-26, 2023*, pp. 611–626. ACM, 2023. doi: 10.1145/3600006.3613165. URL <https://doi.org/10.1145/3600006.3613165>.
- Leviathan, Y., Kalman, M., and Matias, Y. Fast inference from transformers via speculative decoding. In Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., and Scarlett, J. (eds.), *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, Proceedings of Machine Learning Research, pp. 19274–19286. PMLR, 2023. URL <https://proceedings.mlr.press/v202/leviathan23a.html>.
- Li, G., Hammoud, H., Itani, H., Khizbullin, D., and Ghanem, B. CAMEL: communicative agents for “mind” exploration of large language model society. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/a3621ee907def47c1b952ade25c67698-Abstract-Conference.html.
- Li, J., Zhang, Q., Yu, Y., Fu, Q., and Ye, D. More agents is all you need. *Trans. Mach. Learn. Res.*, 2024, 2024a. URL <https://openreview.net/forum?id=bgzUSZ8aeg>.
- Li, X., Wang, S., Zeng, S., Wu, Y., and Yang, Y. A survey on llm-based multi-agent systems: workflow, infrastructure, and challenges. *Vicinagearth*, 1(1):9, 2024b.
- Liu, S., Gu, L., Wei, H., and Basin, D. A. Plume: Efficient and complete black-box checking of weak isolation levels. *Proc. ACM Program. Lang.*, 8(OOPSLA2): 876–904, 2024a. doi: 10.1145/3689742. URL <https://doi.org/10.1145/3689742>.
- Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Yang, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Su,

- Y., Sun, H., Huang, M., Dong, Y., and Tang, J. Agent-bench: Evaluating llms as agents. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024b. URL <https://openreview.net/forum?id=zAdUB0aCTQ>.
- Lowe, R., Wu, Y., Tamar, A., Harb, J., Abbeel, P., and Mordatch, I. Multi-agent actor-critic for mixed cooperative-competitive environments. In Guyon, I., von Luxburg, U., Bengio, S., Wallach, H. M., Fergus, R., Vishwanathan, S. V. N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 6379–6390, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/68a9750337a418a86fe06c1991a1d64c-Abstract.html>.
- Mialon, G., Dessì, R., Lomeli, M., Nalmpantis, C., Pasunuru, R., Raileanu, R., Rozière, B., Schick, T., Dwivedi-Yu, J., Celikyilmaz, A., Grave, E., LeCun, Y., and Scialom, T. Augmented language models: a survey. *Trans. Mach. Learn. Res.*, 2023, 2023. URL <https://openreview.net/forum?id=jh7wH2AzKK>.
- OpenAI. GPT-4 technical report. *CoRR*, abs/2303.08774, 2023. doi: 10.48550/ARXIV.2303.08774. URL <https://doi.org/10.48550/arXiv.2303.08774>.
- Papadimitriou, C. H. The serializability of concurrent database updates. *J. ACM*, 26(4):631–653, 1979. doi: 10.1145/322154.322158. URL <https://doi.org/10.1145/322154.322158>.
- Park, J. S., O’Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., and Bernstein, M. S. Generative agents: Interactive simulacra of human behavior. In Follmer, S., Han, J., Steimle, J., and Riche, N. H. (eds.), *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, UIST 2023, San Francisco, CA, USA, 29 October 2023- 1 November 2023*, pp. 2:1–2:22. ACM, 2023. doi: 10.1145/3586183.3606763. URL <https://doi.org/10.1145/3586183.3606763>.
- Pinecone Systems. Pinecone: The vector database for machine learning. <https://www.pinecone.io/>, 2023.
- Qian, C., Liu, W., Liu, H., Chen, N., Dang, Y., Li, J., Yang, C., Chen, W., Su, Y., Cong, X., Xu, J., Li, D., Liu, Z., and Sun, M. Chatdev: Communicative agents for software development. In Ku, L., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pp. 15174–15186. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.810. URL <https://doi.org/10.18653/v1/2024.acl-long.810>.
- Reed, D. P. *Naming and synchronization in a decentralized computer system*. PhD thesis, Massachusetts Institute of Technology, Cambridge, MA, USA, 1978. URL <https://hdl.handle.net/1721.1/16279>.
- Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. In Oh, A., Naumann, T., Globerson, A., Saenko, K., Hardt, M., and Levine, S. (eds.), *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems 2023, NeurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*, 2023. URL http://papers.nips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html.
- Shapiro, M., Preguiça, N. M., Baquero, C., and Zawirski, M. Conflict-free replicated data types. In Défago, X., Petit, F., and Villain, V. (eds.), *Stabilization, Safety, and Security of Distributed Systems - 13th International Symposium, SSS 2011, Grenoble, France, October 10-12, 2011. Proceedings*, Lecture Notes in Computer Science, pp. 386–400. Springer, 2011. doi: 10.1007/978-3-642-24550-3_29. URL https://doi.org/10.1007/978-3-642-24550-3_29.
- Shavit, N. and Touitou, D. Software transactional memory. In Anderson, J. H. (ed.), *Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computing, Ottawa, Ontario, Canada, August 20-23, 1995*, pp. 204–213. ACM, 1995. doi: 10.1145/224964.224987. URL <https://doi.org/10.1145/224964.224987>.
- Stone, P., Kaminka, G. A., Kraus, S., and Rosenschein, J. S. Ad hoc autonomous agent teams: Collaboration without pre-coordination. In Fox, M. and Poole, D. (eds.), *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2010, Atlanta, Georgia, USA, July 11-15, 2010*, pp. 1504–1509. AAAI Press, 2010. doi: 10.1609/AAAI.V24I1.7529. URL <https://doi.org/10.1609/aaai.v24i1.7529>.
- Tan, C., Zhao, C., Mu, S., and Walfish, M. Cobra: Making transactional key-value stores verifiably serializable. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*, pp. 63–80. USENIX Association, 2020. URL <https://www.usenix.org/conference/osdi20/presentation/tan>.

- Tao, W., Zhou, Y., Wang, Y., Zhang, W., Zhang, H., and Cheng, Y. MAGIS: llm-based multi-agent framework for github issue resolution. In Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J. M., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/5d1f02132ef51602adf07000ca5b6138-Abstract-Conference.html.
- Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Canton-Ferrer, C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., and Scialom, T. Llama 2: Open foundation and fine-tuned chat models. *CoRR*, abs/2307.09288, 2023. doi: 10.48550/ARXIV.2307.09288. URL <https://doi.org/10.48550/arXiv.2307.09288>.
- Tran, K., Dao, D., Nguyen, M., Pham, Q., O’Sullivan, B., and Nguyen, H. D. Multi-agent collaboration mechanisms: A survey of llms. *CoRR*, abs/2501.06322, 2025. doi: 10.48550/ARXIV.2501.06322. URL <https://doi.org/10.48550/arXiv.2501.06322>.
- Wang, G., Xie, Y., Jiang, Y., Mandlkar, A., Xiao, C., Zhu, Y., Fan, L., and Anandkumar, A. Voyager: An open-ended embodied agent with large language models. *Trans. Mach. Learn. Res.*, 2024, 2024a. URL <https://openreview.net/forum?id=ehfRiF0R3a>.
- Wang, Q., Wang, T., Li, Q., Liang, J., and He, B. Megaagent: A practical framework for autonomous cooperation in large-scale LLM agent systems. *CoRR*, abs/2408.09955, 2024b. doi: 10.48550/ARXIV.2408.09955. URL <https://doi.org/10.48550/arXiv.2408.09955>.
- Weaviate. Weaviate: The AI-native vector database. <https://weaviate.io/>, 2023.
- Weikum, G. and Vossen, G. *Transactional Information Systems: Theory, Algorithms, and the Practice of Concurrency Control and Recovery*. Morgan Kaufmann, 2002. ISBN 1-55860-508-8.
- Wu, J. Git context controller: Manage the context of llm-based agents like git. *CoRR*, abs/2508.00031, 2025. doi: 10.48550/ARXIV.2508.00031. URL <https://doi.org/10.48550/arXiv.2508.00031>.
- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Zhang, S., Zhu, E., Li, B., Jiang, L., Zhang, X., and Wang, C. Autogen: Enabling next-gen LLM applications via multi-agent conversation framework. *CoRR*, abs/2308.08155, 2023. doi: 10.48550/ARXIV.2308.08155. URL <https://doi.org/10.48550/arXiv.2308.08155>.
- Yang, X., Li, L., Wuerkaixi, A., Cheng, X., Liu, C., Zeng, K., Cai, X., and Jiang, W. Towards self-robust llms: Intrinsic prompt noise resistance via coipo. *CoRR*, abs/2603.03314, 2026. doi: 10.48550/ARXIV.2603.03314. URL <https://doi.org/10.48550/arXiv.2603.03314>.
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K. R., and Cao, Y. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.
- Yu, C., Velu, A., Vinitisky, E., Gao, J., Wang, Y., Bayen, A. M., and Wu, Y. The surprising effectiveness of PPO in cooperative multi-agent games. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022a. URL http://papers.nips.cc/paper_files/paper/2022/hash/9c1535a02f0ce079433344e14d910597-Abstract-Datasets_and_Benchmarks.html.
- Yu, G., Jeong, J. S., Kim, G., Kim, S., and Chun, B. Orca: A distributed serving system for transformer-based generative models. In Aguilera, M. K. and Weatherspoon, H. (eds.), *16th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2022, Carlsbad, CA, USA, July 11-13, 2022*, pp. 521–538. USENIX Association, 2022b. URL <https://www.usenix.org/conference/osdi22/presentation/yu>.
- Zhang, K., Yang, Z., and Basar, T. Multi-agent reinforcement learning: A selective overview of theories and algorithms. *CoRR*, abs/1911.10635, 2019. URL <http://arxiv.org/abs/1911.10635>.
- Zhang, Y., Liu, F., Shan, Y., Huang, X., Yang, X., Zhu, Y., Cheng, X., Liu, C., Zeng, K., Zhang, T. J., and Jiang, W. Silo-bench: A scalable environment for evaluating

distributed coordination in multi-agent LLM systems. *CoRR*, abs/2603.01045, 2026. doi: 10.48550/ARXIV.2603.01045. URL <https://doi.org/10.48550/arXiv.2603.01045>.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C. W., and Sheng, Y. Sglang: Efficient execution of structured language model programs. In Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J. M., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/724be4472168f31balc9ac630f15dec8-Abstract-Conference.html.

Zhou, S., Xu, F. F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., and Neubig, G. Webarena: A realistic web environment for building autonomous agents. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=oKn9c6ytLx>.

A. LLM Usage

We used large language models (LLMs) solely for writing assistance, including grammar checking, language polishing, and phrasing refinement. All technical content, experimental design, and conclusions are entirely the work of the authors.